

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1521

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I **G. Ramya/192572073** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G. Ramya**

Assessment Tasks

Case Study

1. Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.
2. Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.
3. Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is	Reasonable	Basic solution	Unclear solution

		feasible, practical, and well justified	solution with minor gaps		
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Food Delivery Company – Cloud-Native Backend Architecture

Proposed Architecture:

Browser/Mobile Client → Web/API Layer → Cloud Application Services → Database + Object Storage

Components and Roles

1. Browser/Mobile Clients

- Customers, delivery partners, and restaurants access the application through browsers or mobile apps.
- They send requests such as login, order placement, payment, and delivery tracking.

2. Web/API Layer

- Provides **RESTful web APIs** for communication between clients and backend services.
- Handles authentication, order requests, restaurant information, and delivery status.
- An API gateway can manage routing, security, rate limiting, and request monitoring.

3. Cloud-Native Backend

- Business logic can be divided into services such as:
 - User service
 - Restaurant service
 - Order service
 - Payment service
 - Delivery/tracking service
- Containers or serverless functions can be used to run these services.
- Auto-scaling allows the system to handle peak demand.

4. Cloud Database

- Stores structured information such as customer accounts, restaurant details, orders, payments, and delivery status.
- Managed cloud databases reduce the need for manual database administration.

5. Object Storage

- Stores receipts, food images, delivery photographs, and other large files.
- Object storage is suitable because these files do not need to be accessed like traditional disk blocks.
- It provides scalability, durability, and easy access through APIs.

6. Security and Monitoring

- Authentication, authorization, encryption, firewalls, and secure API access protect the system.
- Monitoring tools track application performance, errors, traffic, and resource utilization.

Conclusion

This architecture provides **scalability, high availability, security, and flexibility**. Web APIs connect clients to cloud services, the backend processes business operations, the database stores structured information, and object storage efficiently handles receipts and delivery images.

Case Study 2: Media Startup – Storage and Serverless Video Processing

The startup has three possible technologies: **SAN-backed storage, object storage, and serverless processing**.

1. SAN-Backed Storage

- SAN provides high-performance block-level storage.
- It is useful when applications require frequent and low-latency block-level access.
- However, it can be expensive and requires more infrastructure management.
- It is less suitable for storing a rapidly growing collection of independent video files.

2. Object Storage

- Object storage is highly suitable for **video uploads**.
- Each video can be stored as an object with metadata.
- It provides high scalability and durability.
- Storage capacity can grow without the startup having to manage physical storage infrastructure.

- Videos can be accessed through APIs and integrated easily with cloud services.

3. Serverless Processing

- After a video is uploaded, an event can automatically trigger a serverless function or processing service.
- The video can be:
 - Transcoded into different resolutions
 - Compressed
 - Given thumbnails
 - Analyzed for metadata
- The startup pays mainly for the processing resources used rather than maintaining continuously running servers.

Recommended Deployment

Object Storage + Serverless Processing is the best choice.

Workflow:

Video Upload → Object Storage → Upload Event → Serverless Processing → Processed Video/Thumbnail → Object Storage → CDN/User

Why This Is Recommended

Requirement	SAN	Object Storage	Serverless
Large video storage	Moderate	Excellent	Not applicable
Scalability	Moderate	Excellent	Excellent
Infrastructure management	High	Low	Very low
Video processing	Good	Storage role	Excellent
Cost efficiency for variable workloads	Moderate	High	High

Conclusion

The startup should use **object storage as the primary video repository and serverless processing for event-driven video workflows**. SAN can be considered only when specialized applications require high-performance

block-level storage. This deployment reduces infrastructure management and handles unpredictable video-processing workloads efficiently.

Case Study 3: Smart City – Edge and Fog Computing

A smart city may have thousands of sensors generating data from traffic systems, surveillance cameras, environmental monitors, smart buildings, and public infrastructure. Sending every piece of data directly to a distant cloud can introduce latency and consume significant bandwidth.

Edge Computing

Edge computing processes data close to the sensors or devices that generate it.

For example:

- Traffic cameras can detect accidents locally.
- Traffic signals can immediately respond to congestion.
- Sensors can detect dangerous environmental conditions.
- Emergency systems can make rapid decisions without waiting for a cloud response.

Advantages:

- Very low latency
- Faster decision-making
- Reduced bandwidth consumption
- Continued operation even when cloud connectivity is temporarily unavailable
- Improved privacy because some data can be processed locally

Fog Computing

Fog computing provides an intermediate computing layer between edge devices and the central cloud.

For example:

Sensors → Edge Devices → Fog/Local Gateway → Cloud

Fog nodes can collect and combine data from multiple nearby sensors before sending important information to the cloud.

They can perform:

- Data filtering
- Aggregation
- Local analytics
- Temporary storage
- Real-time coordination between nearby devices

Role of Cloud Computing

The cloud remains responsible for tasks that do not require immediate responses, such as:

- Long-term data storage
- Historical analysis
- Machine learning model training
- City-wide analytics
- Generating reports and dashboards
- Centralized management

Recommended Architecture

Sensors → Edge Layer → Fog Layer → Cloud Layer

For example, in a smart traffic system:

Traffic Sensors/Cameras → Edge Processing → City-Level Fog Gateway → Cloud Analytics

If an accident is detected, the **edge layer** can immediately identify it and adjust nearby signals. The **fog layer** can coordinate traffic information across several intersections, while the **cloud** can analyze months of traffic data to improve future traffic planning.

Conclusion

A **hybrid Edge–Fog–Cloud architecture** is the most appropriate solution for a smart city. Edge computing provides **ultra-low-latency decisions**, fog computing provides **local coordination and aggregation**, and cloud computing provides **large-scale storage and advanced analytics**. This combination improves response time, reduces network traffic, and provides a scalable architecture for smart-city services.

